PDS TECH

DEVOPS E ARQUITETURA CLOUD

FASE 4 | AULA 06 -

NEW RELIC - INSTRUMENTANDO APM

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SUMÁRIO

O QUE VEM POR AI? 3
HANDS ON 4
SAIBA MAIS 5
O QUE VOCÊ VIU NESTA AULA? 31
REFERÊNCIAS 32

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VEM POR AI?

Esta aula abordará os fundamentos e práticas de **instrumentação de Application Performance Monitoring (APM) utilizando o New Relic**, com foco na compreensão profunda de como aplicações modernas podem ser tornadas observáveis desde a camada de código até suas dependências externas.

O conteúdo explorará a arquitetura da plataforma New Relic, o papel dos agentes de instrumentação, o funcionamento do ciclo de coleta de dados e o modelo de armazenamento baseado em telemetria. Ao longo da aula, serão apresentados os conceitos centrais que sustentam o APM moderno, preparando o terreno para análises mais avançadas de performance, confiabilidade e comportamento de sistemas distribuídos em ambientes cloud-native.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

HANDS ON

A parte prática da aula será dedicada à **instrumentação real de uma aplicação .NET com o agente APM do New Relic**, permitindo observar, na prática, como os dados de performance são coletados, processados e disponibilizados na plataforma.

Serão realizadas atividades de configuração do agente, geração de tráfego, análise de transações, inspeção de traces e aplicação de instrumentação customizada. Além disso, serão exploradas as visualizações de entidades e relacionamentos, evidenciando como a observabilidade emerge a partir da correta modelagem da aplicação e de suas interações.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

SAIBA MAIS

Fundamentos e Configuração Inicial

Criando uma Conta no New Relic

A criação de uma conta no New Relic é o primeiro passo para a instrumentação e monitoramento de aplicações. A plataforma oferece diferentes planos, cada um desenhado para atender a necessidades específicas, desde pequenos projetos até grandes corporações. A escolha do plano impacta diretamente a quantidade de dados que podem ser ingeridos, o período de retenção desses dados, o número de usuários e o acesso a funcionalidades avançadas.

Planos de Assinatura:

- **Free Tier (Grátis):** ideal para indivíduos e pequenas equipes que estão começando a explorar a observabilidade. Oferece 100 GB de ingestão de dados por mês e um usuário "Full Platform". É uma excelente maneira de experimentar a maioria das funcionalidades sem custo.
- **Standard:** destinado a equipes que necessitam de uma colaboração mais ampla, permitindo múltiplos usuários "Full Platform" e um volume maior de ingestão de dados. O preço é baseado no volume de dados e no número de usuários.
- **Pro:** voltado para organizações que necessitam de funcionalidades avançadas de segurança, governança e suporte técnico especializado. Inclui tudo do plano Standard, com adicionais como logon único (SSO) e limites de retenção de dados mais elevados.
- **Enterprise:** a solução mais completa, desenhada para grandes corporações com requisitos complexos de segurança, escalabilidade e suporte premium. Oferece opções de personalização e um gerente de contas dedicado.

Estrutura Organizacional

O New Relic organiza as contas em uma hierarquia que permite a gestão centralizada de múltiplos ambientes e equipes. A estrutura básica consiste em uma

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Organização, que pode conter múltiplas **Contas**. Cada conta é um contêiner isolado para dados, usuários e configurações. Essa estrutura é particularmente útil para empresas que precisam separar ambientes de desenvolvimento, teste e produção, ou para consultorias que gerenciam múltiplos clientes.

Nível Hierárquico	Descrição
Organização	O nível mais alto da hierarquia, que agrupa todas as contas de uma empresa.
Conta	Um ambiente isolado para dados, usuários, chaves de API e configurações. Uma organização pode ter várias contas.
Usuário	Associado a uma ou mais contas dentro da organização, com permissões específicas.

Figura 1 - Estrutura Organizacional
Fonte: Elaborado pelo autor (2026)

Estrutura de Contas, Users e Api Keys

A gestão eficaz de acesso e segurança é um pilar fundamental da observabilidade. No New Relic, isso é alcançado através de uma estrutura bem definida de contas, usuários e chaves de API. Compreender como esses elementos se inter-relacionam é crucial para manter a integridade e a segurança dos dados de telemetria.

Usuários e Papéis (Users and Roles):

O New Relic utiliza um sistema de controle de acesso baseado em papéis (RBAC - Role-Based Access Control). Os usuários são adicionados a uma organização e podem ter acesso a uma ou mais contas. Para cada conta, um usuário é associado a um papel que define seu nível de permissão.

Os papéis padrão incluem:

- **Admin:** possui controle total sobre a conta, incluindo configurações, gestão de usuários e dados.
- **User:** pode visualizar dashboards, analisar dados e configurar alertas, mas não pode alterar configurações da conta ou gerenciar usuários.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Read-only:** permissão apenas para visualização de dados, sem capacidade de criar ou modificar alertas, dashboards ou configurações.

Além dos papéis padrão, os planos Pro e Enterprise permitem a criação de papéis customizados, oferecendo um controle de acesso ainda mais granular.

As chaves de API são essenciais para a automação e integração do New Relic com outras ferramentas e para o envio de dados para a plataforma. Existem diferentes tipos de chaves, cada uma com um propósito específico:

- **Ingest Keys (Chaves de Ingestão):** usadas para enviar dados de telemetria (métricas, eventos, logs, traces) para o New Relic. A principal chave deste tipo é a **License Key**, que é usada pela maioria dos agentes de instrumentação.
- **User Keys (Chaves de Usuário):** associadas a um usuário específico, herdam as permissões desse usuário. São usadas para interagir com a API NerdGraph (GraphQL) e APIs REST para consultar dados, gerenciar recursos (como dashboards e alertas) e automatizar configurações.
- **Browser Keys (Chaves de Browser):** usadas especificamente pelo agente de monitoramento de Browser para reportar dados de performance do front-end.

A gestão cuidadosa das chaves de API é vital. Elas devem ser tratadas como senhas, armazenadas de forma segura e rotacionadas periodicamente. É uma boa prática criar chaves de usuário específicas para cada automação ou integração, com o mínimo de privilégios necessários, em vez de usar a chave de um usuário administrador.

Tipo de Chave	Propósito Principal	Exemplo de Uso
License Key	Enviar dados de telemetria para o New Relic.	Configuração do agente .NET no `newrelic.config`.
User Key	Consultar dados e gerenciar recursos via API.	Scripts de automação para criar dashboards ou políticas de alerta.
Browser Key	Coletar dados de performance do front-end.	Injeção do script do agente Browser em uma página web.

Figura 2 – Chaves de API
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Arquitetura do New Relic

Para extrair o máximo valor da plataforma New Relic, é crucial entender sua arquitetura subjacente. O New Relic não é uma caixa preta; ele é um sistema distribuído complexo, projetado para coletar, processar e analisar volumes massivos de dados de telemetria em tempo real. A arquitetura é construída em torno de quatro pilares principais: **Agentes de Dados**, a **Rede de Ingestão**, o **Banco de Dados de Telemetria (NRDB)** e a **Plataforma de Aplicações**.

Agentes de Dados (Data Agents): os agentes são pequenas bibliotecas de código que são instaladas em sua aplicação ou infraestrutura. Eles são responsáveis por coletar os dados de telemetria. Para uma aplicação .NET, o agente APM se anexa ao processo do CLR (Common Language Runtime) e usa técnicas de profiling para interceptar chamadas de método, queries a bancos de dados, chamadas a serviços externos, etc. Esses dados brutos são então agregados e enviados para o New Relic em intervalos regulares, tipicamente a cada minuto (conhecido como "harvest cycle").

Rede de Ingestão (Ingestion Network): uma vez que os agentes enviam os dados, eles chegam à rede de ingestão global e altamente escalável do New Relic. Esta rede é um conjunto de endpoints distribuídos geograficamente que recebem os dados dos agentes. A primeira tarefa da rede de ingestão é autenticar os dados usando a License Key e, em seguida, processá-los e enriquecê-los. Por exemplo, adicionando metadados como o ID da conta, nome da aplicação, etc. Os dados são então normalizados em um formato comum antes de serem enviados para o banco de dados.

Banco de Dados de Telemetria (NRDB - New Relic Database): o coração da plataforma é o NRDB, um banco de dados de série temporal massivamente paralelo e customizado, projetado especificamente para armazenar e consultar dados de telemetria. O NRDB é otimizado para consultas analíticas em alta velocidade sobre grandes volumes de dados. Todos os tipos de dados — Métricas, Eventos, Logs e Traces (MELT) — são armazenados no NRDB. É o poder do NRDB que permite a execução de consultas complexas via NRQL (New Relic Query Language) com tempos de resposta de segundos, mesmo sobre petabytes de dados.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Plataforma de Aplicações (Application Platform): no topo da arquitetura está a plataforma de aplicações, que inclui a interface do usuário (UI) do New Relic, o sistema de alertas, os dashboards e a API NerdGraph. Esta camada consome os dados do NRDB e os apresenta de forma visual e interativa para os usuários. Quando você visualiza um gráfico de tempo de resposta no APM ou executa uma query NRQL, é a plataforma de aplicações que está consultando o NRDB e renderizando os resultados.

Fluxo de Dados Simplificado:

Agente (.NET) -> Rede de Ingestão -> NRDB -> UI/Alerts/API

Esta arquitetura desacoplada e baseada em nuvem permite que o New Relic ofereça uma plataforma robusta, escalável e resiliente, capaz de monitorar desde uma única aplicação até ecossistemas de microsserviços complexos distribuídos globalmente.

Instrumentação de Aplicações e Transações

Conceito de Transações

No universo do New Relic, o conceito de **transação** é a unidade fundamental de trabalho e a base para a análise de performance. Uma transação representa uma única operação lógica do ponto de vista da aplicação. O New Relic APM mede o tempo total gasto nessa operação e detalha o tempo consumido em cada um de seus componentes internos, como chamadas a bancos de dados, serviços externos e o próprio código da aplicação. Entender e classificar corretamente as transações é o primeiro passo para identificar gargalos de performance e otimizar a experiência do usuário.

As transações são divididas em duas categorias principais:

Web Transactions (Transações Web): são as transações iniciadas por uma requisição web externa, tipicamente uma requisição HTTP. Elas representam a interação direta de um usuário ou de outro sistema com a aplicação. Para a maioria das organizações, estas são as transações mais críticas a serem monitoradas, pois impactam diretamente a experiência do cliente.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Exemplos:

- Carregamento de uma página em um site de e-commerce.
- Submissão de um formulário de login.
- Chamada de API a partir de uma aplicação mobile para buscar dados.
- Um webhook recebido de um serviço de terceiros.

O New Relic automaticamente nomeia essas transações com base no framework utilizado. Em uma aplicação ASP.NET Core, por exemplo, o nome da transação geralmente segue o padrão Controller/Action.

Background Transactions (Transações de Background ou Non-Web): são transações que não são iniciadas por uma requisição web. Elas representam trabalhos que a aplicação executa em segundo plano, como processamento de filas, tarefas agendadas, e outras operações assíncronas que não estão diretamente ligadas a uma interação de usuário em tempo real.

Exemplos:

- Processamento de um item de uma fila de mensagens (RabbitMQ, Azure Service Bus).
- Execução de uma tarefa agendada (cron job) para gerar relatórios noturnos.
- Um processo de ETL (Extract, Transform, Load) que roda em segundo plano.
- Operações de longa duração iniciadas por um console application.

Monitorar transações de background é crucial para a saúde do sistema como um todo, pois problemas de performance nessas áreas podem causar atrasos em processos de negócio, acúmulo de filas e consumo excessivo de recursos.

O New Relic APM apresenta dados agregados para cada tipo de transação em sua própria página, permitindo analisar a performance, o throughput (requisições por minuto) e a taxa de erros de forma isolada e comparativa.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Tipo de Transação	Início da Operação	Exemplo Típico em .NET	Criticidade
Web Transaction	Requisição HTTP	Chamada a um endpoint de uma Web API.	Alta (impacto direto no usuário)
Background Transaction	Não-HTTP (ex: job, mensagem)	Processamento de uma mensagem com `IHostedService`.	Média a Alta (impacto em processos de negócio)

Figura 3 – Transações de Background
Fonte: Elaborado pelo autor (2026)

Instrumentação Automática x Manual

A beleza do New Relic APM reside em sua capacidade de fornecer insights valiosos com o mínimo de configuração. Isso é possível graças à **instrumentação automática**, mas sua verdadeira força é desbloqueada quando combinada com a **instrumentação manual** (ou customizada).

Instrumentação Automática (Auto-Instrumentation): quando você instala o agente APM em uma aplicação que usa um framework suportado (como ASP.NET, ASP.NET Core, WCF, etc.), o agente automaticamente detecta e instrumenta as partes mais importantes do seu código. Ele faz isso "se anexando" ao runtime da sua aplicação (o CLR no caso do .NET) e envolvendo métodos conhecidos para medir sua performance.

O que é tipicamente instrumentado automaticamente:

- **Requisições Web:** o ciclo de vida completo de uma requisição HTTP, desde o recebimento até o envio da resposta.
- **Chamadas a Banco de Dados:** queries executadas através de bibliotecas de acesso a dados populares (Entity Framework, Dapper, System.Data.SqlClient). O agente captura o tipo de operação (SELECT, INSERT, etc.) e o tempo de execução. Por segurança, os parâmetros da query não são capturados por padrão.
- **Chamadas a Serviços Externos:** requisições HTTP feitas pela sua aplicação para outras APIs ou serviços (usando HttpClient, por exemplo).
- **Componentes do Framework:** middlewares, roteamento, execução de controllers e actions em ASP.NET Core.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O principal benefício da instrumentação automática é o **valor imediato**. Sem alterar uma única linha de código, você obtém uma visão geral da performance da sua aplicação, identifica as transações mais lentas e vê onde o tempo está sendo gasto.

Instrumentação Manual (Custom Instrumentation): a instrumentação automática é poderosa, mas não pode ver tudo. Há cenários em que você precisa dizer explicitamente ao New Relic o que monitorar. É aqui que entra a instrumentação manual.

Quando usar instrumentação manual:

- **Código não coberto:** para medir a performance de um método específico ou de uma biblioteca que não é suportada nativamente pelo agente.
- **Transações de Background:** para definir o início e o fim de uma tarefa em segundo plano que não é uma requisição web.
- **Melhorar a Nomenclatura:** para dar nomes mais significativos às transações do que os nomes padrão gerados pelo agente.
- **Adicionar Contexto:** para anexar dados de negócio (atributos customizados) a uma transação, como ID do usuário ou SKU do produto, permitindo análsies mais ricas.

A abordagem ideal é começar com a instrumentação automática para obter uma linha de base e, em seguida, usar a instrumentação manual de forma estratégica para aprofundar a análise nas áreas mais críticas ou complexas da sua aplicação.

Característica	Instrumentação Automática	Instrumentação Manual
Esforço	Baixo (instalar e configurar o agente)	Médio a Alto (requer modificação no código ou XML)
Visibilidade	Boa (para frameworks suportados)	Excelente (visibilidade em qualquer parte do código)
Flexibilidade	Baixa (limitada ao que o agente suporta)	Alta (controle total sobre o que é medido)
Cenário Ideal	Obter uma visão geral rápida da aplicação.	Aprofundar em gargalos específicos, monitorar código customizado.

Figura 4 – Instrumentação Automática x Manual
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Custom Instrumentation

A instrumentação customizada (ou manual) é a ferramenta que nos permite estender a visibilidade do New Relic para além do que é capturado automaticamente. Ela nos dá o poder de transformar "tempo não instrumentado" em segmentos claros e nomeados, e de monitorar lógicas de negócio que o agente não poderia conhecer. No .NET, existem duas abordagens principais para a instrumentação customizada: **Atributos no Código** e **Configuração via XML**.

Instrumentação via Atributos (Attribute-based Instrumentation): esta é a abordagem mais comum e flexível. Ela envolve decorar seus métodos .NET com atributos especiais fornecidos pela API do agente New Relic. É simples, explícita e viaja junto com o seu código-fonte.

- **Transaction**: este atributo transforma um método em uma nova transação. É usado para instrumentar código que não é automaticamente capturado como uma transação, como um método de entrada em um Console App ou uma tarefa em background. Você pode especificar se é uma transação Web ou Default (non-web).
- **Trace**: este atributo transforma um método em um **segmento** dentro de uma transação já existente. É a ferramenta perfeita para adicionar detalhe a uma transação, quebrando um bloco grande de "tempo gasto no código" em pedaços menores e mais compreensíveis. Por exemplo, você pode usar [Trace] em métodos que realizam cálculos complexos, manipulam arquivos ou fazem lógica de negócio intensiva.

Instrumentação via XML (XML-based Instrumentation): nesta abordagem, você cria um arquivo XML que especifica quais assemblies, classes e métodos devem ser instrumentados. O agente lê este arquivo na inicialização e aplica a instrumentação dinamicamente, sem que você precise alterar seu código-fonte ou ter uma dependência da API do New Relic.

- **Vantagens:** não requer modificação do código-fonte, o que pode ser útil para instrumentar bibliotecas de terceiros ou código legado onde a alteração não é uma opção.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Desvantagens:** é mais frágil. Se o nome de um método, classe ou assembly mudar, a instrumentação quebra silenciosamente. É também mais verboso e complexo de configurar do que os atributos.

Instrumentação via XML (XML-based Instrumentation): nesta abordagem, você cria um arquivo XML que especifica quais assemblies, classes e métodos devem ser instrumentados. O agente lê este arquivo na inicialização e aplica a instrumentação dinamicamente, sem que você precise alterar seu código-fonte ou ter uma dependência da API do New Relic.

- **Vantagens:** não requer modificação do código-fonte, o que pode ser útil para instrumentar bibliotecas de terceiros ou código legado onde a alteração não é uma opção.
- **Desvantagens:** é mais frágil. Se o nome de um método, classe ou assembly mudar, a instrumentação quebra silenciosamente. É também mais verboso e complexo de configurar do que os atributos.

Além dos atributos, o agente .NET expõe uma API completa (NewRelic.Api.Agent.NewRelic) que oferece controle programático total. Com a API, você pode:

- **RecordMetric():** enviar métricas customizadas (ex: número de itens em um carrinho de compras).
- **RecordCustomEvent():** enviar eventos customizados com dados de negócio ricos.
- **NoticeError():** reportar erros que não foram automaticamente capturados.
- **AddCustomParameter():** adicionar atributos (parâmetros) a uma transação existente (ex: userId).

Método	Vantagem Principal	Desvantagem Principal	Caso de Uso
Atributos	Simples, explícito, viaja com o código.	Requer modificação no código.	Adicionar detalhes a métodos específicos.
XML	Não modifica o código-fonte.	Frágil a refatorações, complexo.	Instrumentar código de terceiros.
API	Controle programático total.	Mais verboso para instrumentação simples.	Enviar dados customizados (métricas, eventos).

Figura 5 – Custom Instrumentation

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Fonte: Elaborado pelo autor (2026)

Entidades e Conceitos Fundamentais

Entendimento do Modelo (Tipos de Entidades)

O modelo de dados do New Relic é construído sobre o conceito de **entidade**. Uma entidade é, de forma ampla, qualquer coisa que você monitora. Pode ser um serviço, um host, um contêiner Docker, uma função Lambda, um banco de dados, ou até mesmo um agrupamento lógico customizado, como um dashboard ou um workload. Cada entidade possui um identificador único, o entityGuid, que serve como sua identidade primária em toda a plataforma.

O propósito deste modelo é ir além de silos de dados (métricas de APM em um lugar, dados de infraestrutura em outro) e criar um grafo de conhecimento sobre o seu sistema. Ao entender que uma aplicação (entidade APM) *roda em* um host (entidade de infraestrutura) que, por sua vez, *chama* um banco de dados (outra entidade), o New Relic pode fornecer um contexto muito mais rico durante a análise de problemas.

O New Relic define uma vasta gama de tipos de entidade, que são automaticamente identificados pelos agentes e integrações.

- **APM Application:** uma aplicação ou serviço instrumentado pelo agente APM. Esta é a entidade central para o monitoramento de performance de código.
- **Host:** uma máquina física ou virtual monitorada pelo agente de Infraestrutura. Fornece métricas de CPU, memória, disco e rede.
- **Database:** um serviço de banco de dados. Pode ser monitorado através do agente APM (capturando chamadas da aplicação) ou por integrações de infraestrutura (capturando métricas internas do banco).
- **Browser Application:** uma aplicação front-end monitorada pelo agente Browser. Captura dados de performance da perspectiva do usuário final.
- **Mobile Application:** uma aplicação móvel (iOS/Android) monitorada pelo agente Mobile.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Kubernetes Cluster/Node/Pod/Container:** entidades que representam os diferentes níveis de um ambiente Kubernetes, fornecendo uma visão detalhada da saúde do cluster.
- **AWS/Azure/GCP Services:** centenas de entidades que representam serviços de nuvem, como S3 Buckets, RDS Instances, VMs, etc., coletadas através de integrações com os provedores de nuvem.
- **Workload:** uma entidade customizada que agrupa outras entidades. Permite que você organize seus recursos de uma forma que faça sentido para o seu negócio (ex: agrupar todos os serviços e hosts que compõem o "Sistema de Faturamento").

A verdadeira força do modelo está nos relacionamentos que o New Relic estabelece automaticamente entre as entidades. Esses relacionamentos formam um mapa de como os componentes do seu sistema interagem.

- **RUNS_ON**: uma aplicação APM *roda em* um Host.
- **CALLS:** uma aplicação APM *chama* outra aplicação APM ou serviço externo.
- **CONTAINS**: um Host *contém* um Contêiner Docker.

Esses relacionamentos são os que permitem, por exemplo, que ao analisar uma transação lenta, você possa pular diretamente para as métricas de CPU do host em que a aplicação estava rodando naquele momento, ou para os detalhes da aplicação externa que ela chamou.

Compreender que tudo no New Relic é uma entidade com metadados e relacionamentos é a chave para navegar na plataforma de forma eficiente e realizar análises de causa raiz complexas.

Tipo de Entidade	Fonte de Dados Principal	Exemplo de Relacionamento
APM Application	Agente New Relic APM	`CALLS` outra APM Application
Host	Agente New Relic Infrastructure	`CONTAINS` um Docker Container
Database	Integração de Infraestrutura	É chamado por (`IS_CALLED_BY`) uma APM Application
Workload	Definição do Usuário	`GROUPS` múltiplas entidades

Figura 6 – Entidades

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Fonte: Elaborado pelo autor (2026)

Entity Explorer

O **Entity Explorer** é a porta de entrada principal para a observabilidade no New Relic. É a interface que permite navegar, pesquisar, filtrar e inspecionar todas as entidades monitoradas em sua conta. Pense nele como um inventário completo e em tempo real de toda a sua infraestrutura e aplicações. A página inicial do New Relic, **All entities**, é a manifestação principal do Entity Explorer.

Visão Unificada: o Explorer agrega entidades de todas as fontes (APM, Infraestrutura, Browser, Mobile, Integrações de Nuvem, etc.) em um único local. Isso quebra os silos de monitoramento e permite uma visão holística do sistema.

Busca e Filtragem Poderosa: a funcionalidade mais importante do Explorer é sua capacidade de filtrar e facetar as entidades com base em seus metadados e tags. Você pode rapidamente encontrar o que procura, mesmo em ambientes com milhares de entidades. Por exemplo, você pode filtrar por:

- tags.environment = 'production' para ver apenas entidades de produção.
- entityType = 'HOST' para listar todos os hosts.
- alertSeverity != 'NOT_ALERTING' para encontrar todas as entidades que estão atualmente em um estado de alerta.

Status de Saúde (Health Status): cada entidade no Explorer é visualmente decorada com seu status de saúde atual, geralmente indicado por cores (verde, amarelo, vermelho). Isso permite identificar rapidamente quais partes do seu sistema estão com problemas. O status é determinado pelas políticas de alerta configuradas para aquela entidade.

Contexto e Navegação: a partir do Explorer, você pode clicar em qualquer entidade para mergulhar em seus detalhes. Por exemplo, clicar em uma entidade de aplicação APM leva você ao dashboard de APM, enquanto clicar em um host leva à sua página de métricas de infraestrutura. Ele serve como um ponto de partida para investigações mais profundas.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Vistas Salvas (Saved Views): você pode salvar conjuntos de filtros complexos como uma "vista". Isso é extremamente útil para equipes que frequentemente precisam olhar para o mesmo subconjunto de entidades. Por exemplo, a equipe de banco de dados pode ter uma vista salva que mostra todas as entidades do tipo DATABASE no ambiente de produção.

O Entity Explorer não é apenas uma lista de coisas que você monitora; é uma ferramenta de diagnóstico interativa. Ao dominar a arte de filtrar e navegar no Explorer, você pode reduzir drasticamente o tempo necessário para identificar a causa raiz de um problema, movendo-se de uma visão de alto nível do sistema para os detalhes de uma entidade específica em poucos cliques.

Funcionalidade	Descrição	Benefício Principal
Visão Unificada	Agrega todas as entidades em um só lugar.	Quebra silos de monitoramento.
Busca e Filtragem	Permite encontrar entidades por tags e metadados.	Localização rápida de recursos em ambientes complexos.
Status de Saúde	Mostra o status de alerta de cada entidade.	Identificação imediata de componentes com problemas.
Vistas Salvas	Permite salvar e compartilhar conjuntos de filtros.	Acelera o acesso a subconjuntos de sistemas relevantes para equipes.

Figura 7 – Entity Explorer
Fonte: Elaborado pelo autor (2026)

Entity Navigator e o Lookout

Enquanto o Entity Explorer oferece uma visão de inventário detalhada, o New Relic fornece duas ferramentas visuais de alto nível, o **Navigator** e o **Lookout**, para ajudar a entender a saúde e o comportamento do seu sistema de forma mais intuitiva e em larga escala. Ambas as ferramentas são acessíveis a partir da página principal **All entities** e são projetadas para responder a perguntas diferentes sobre o seu ecossistema.

O Navigator oferece uma visão panorâmica de todo o seu sistema em uma única tela. Ele representa suas entidades como um "favo de mel" (honeycomb) de hexágonos, onde cada hexágono é uma entidade. A cor de cada hexágono representa

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

o status de saúde (alerta) daquela entidade, usando a mesma codificação de cores (verde, amarelo, vermelho) do resto da plataforma.

Propósito: responder à pergunta "Como está a saúde geral do meu sistema agora?".

Visualização: alta densidade de informação. Permite ver milhares de entidades de uma só vez.

Interatividade: você pode agrupar os hexágonos por atributos ou tags (ex: agrupar por aws.region para ver a saúde por região da nuvem) e filtrar para focar em partes específicas do sistema. Clicar em um hexágono revela um resumo dos detalhes e métricas daquela entidade.

Caso de Uso: ideal para um NOC (Network Operations Center) ou para uma equipe de SRE (Site Reliability Engineering) que precisa de uma visão "olho de pássaro" constante da saúde de todo o ambiente. Se uma área do favo de mel ficar vermelha, indica um problema localizado que precisa de atenção imediata.

O Lookout adota uma abordagem diferente. Em vez de mostrar o status de saúde atual, ele mostra o **desvio do comportamento normal**. Ele usa análise estatística para comparar o desempenho recente de uma entidade (últimos 5 minutos) com seu desempenho em um período anterior (última hora) e destaca as entidades que estão se comportando de forma anômala.

Propósito: responder à pergunta "O que mudou ou está se comportando de forma estranha no meu sistema?".

Visualização: as entidades são representadas como círculos. O **tamanho** do círculo indica a magnitude da mudança (círculos maiores = maiores desvios). A **cor** do círculo indica a severidade da mudança (ex: vermelho para um grande aumento na taxa de erros).

Análise Automática: o Lookout analisa automaticamente os "golden signals" (throughput, tempo de resposta e erros) para cada tipo de entidade, sem a necessidade de configurar alertas.

Caso de Uso: perfeito para a detecção proativa de problemas. Mesmo que um alerta ainda não tenha sido disparado, o Lookout pode mostrar que o tempo de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

resposta de um serviço está começando a aumentar, permitindo que a equipe investigue antes que o problema impacte os usuários.

Juntos, o Navigator e o Lookout fornecem duas perspectivas complementares e poderosas. O Navigator é reativo ao estado de alerta que você definiu, enquanto o Lookout é proativo, encontrando problemas que você talvez nem soubesse que precisava procurar.

Ferramenta	Pergunta que Responde	Visualização	Base da Análise
Navigator	"Como está a saúde do meu sistema?"	Favo de mel (Hexágonos)	Status de Alerta (Configurado)
Lookout	"O que está se comportando de forma estranha?"	Círculos	Desvio do Comportamento Normal (Automático)

Figura 8 – Navigator/ Lookout
Fonte: Elaborado pelo autor (2026)

Service Map

Em uma arquitetura moderna, especialmente uma baseada em microsserviços, entender como os diferentes componentes se conectam e se comunicam é tão importante quanto monitorar os componentes individualmente. O **Service Map** do New Relic é a ferramenta que fornece essa visão de dependências, desenhando um mapa visual e dinâmico das relações entre seus serviços, bancos de dados e serviços externos.

O Service Map é uma representação gráfica do fluxo de chamadas entre as entidades. Ele mostra:

Nós (Nodes): cada serviço, banco de dados ou serviço externo é representado como um nó no mapa.

Conexões (Edges): as linhas que conectam os nós representam as chamadas entre eles. Se o Serviço A faz uma chamada de API para o Serviço B, haverá uma conexão do nó A para o nó B.

Saúde e Performance: a cor dos nós e das conexões indica a saúde (baseada em alertas) e a performance (tempo de resposta e taxa de erros). Uma conexão

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

vermelha entre dois serviços indica que as chamadas entre eles estão falhando ou estão excessivamente lentas.

Direção do Tráfego: as setas nas conexões mostram a direção do fluxo de chamadas, deixando claro quem é o chamador e quem é o chamado.

O Service Map é alimentado principalmente pelos dados de **Distributed Tracing**. Quando uma requisição atravessa múltiplos serviços instrumentados, o agente do New Relic propaga um cabeçalho de contexto em cada chamada. Esse cabeçalho permite que o New Relic "costure" as diferentes partes da requisição em um único trace distribuído. Ao analisar milhares desses traces, o New Relic constrói o mapa de quais serviços se comunicam entre si.

Mesmo sem o Distributed Tracing habilitado, o Service Map ainda pode inferir algumas conexões (como chamadas a bancos de dados e serviços externos a partir de um único serviço), mas a visão de ponta a ponta através de múltiplos serviços instrumentados depende do Distributed Tracing. Benefícios do Service Map:

Descoberta de Arquitetura: para equipes que herdam sistemas complexos ou para arquiteturas que evoluem rapidamente, o Service Map serve como uma documentação viva e automática da arquitetura de serviços.

Análise de Impacto: quando um serviço está com problemas (ex: Serviço de Autenticação), o mapa mostra imediatamente quais serviços dependem dele (upstream) e quais serviços ele chama (downstream). Isso ajuda a entender o raio de impacto de uma falha.

Identificação de Gargalos: o mapa destaca visualmente as conexões lentas ou com erros, apontando diretamente para os pontos de integração que estão degradando a performance do sistema.

Detecção de Dependências Inesperadas: o mapa pode revelar comunicações entre serviços que não eram esperadas pela equipe de desenvolvimento, ajudando a identificar acoplamentos indesejados ou falhas de design.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Funcionalidade	Descrição	Caso de Uso
Visualização de Topologia	Desenha um mapa de nós e conexões.	Entender a arquitetura de um sistema de microsserviços.
Colorização por Saúde	Colore nós e conexões com base em performance e alertas.	Identificar rapidamente onde está a origem de uma lentidão.
Análise de Dependências	Mostra serviços upstream e downstream.	Avaliar o impacto de colocar um serviço em manutenção.
Dados de Conexão	Mostra throughput e tempo de resposta por conexão.	Diagnosticar se um problema está no serviço A, no serviço B ou na rede entre eles.

Figura 9 – Service Map
Fonte: Elaborado pelo autor (2026)

Tagging e Metadata

Em ambientes de qualquer tamanho, mas especialmente nos grandes e complexos, a capacidade de organizar, fatiar e filtrar dados é fundamental. O New Relic resolve esse problema através de um sistema robusto de **tags** e **metadados**. Tags são pares de chave-valor (ex: environment:production) que você pode anexar a qualquer entidade, permitindo que você adicione contexto de negócio e organizacional aos seus dados de telemetria.

Uma tag é simplesmente um rótulo. Ela não altera o comportamento da entidade, mas enriquece seus dados, tornando-os mais pesquisáveis e fáceis de agrupar. O New Relic automaticamente adiciona várias tags a cada entidade, como:

- accountId: o ID da conta New Relic.
- appName: o nome da aplicação.
- host: o nome do host.
- language: a linguagem da aplicação (ex: dotnet).
- Tags de provedores de nuvem (ex: aws.region, azure.resourceGroup).

Além das tags automáticas, você pode (e deve) adicionar suas próprias tags customizadas para refletir a estrutura da sua organização. Por que usar Tags?

Filtragem e Busca: a principal utilidade das tags é filtrar dados no Entity Explorer, Navigator e em todo o New Relic. Você pode rapidamente isolar entidades

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

que pertencem a um ambiente específico, a uma equipe, a um data center ou a um cliente. *Exemplo: Encontrar todos os serviços da equipe 'alpha' no ambiente de produção: tags.team = 'alpha' AND tags.environment = 'production'*

Agrupamento e Agregação: As tags permitem agrupar dados em dashboards e queries NRQL. Você pode comparar a performance de diferentes ambientes, equipes ou versões de uma aplicação. *Exemplo: Comparar o tempo de resposta médio por ambiente: SELECT average(duration) FROM Transaction FACET tags.environment*

Governança e Responsabilidade: As tags ajudam a identificar quem é o "dono" de um serviço ou recurso. Quando um alerta dispara, uma tag team pode indicar imediatamente qual equipe precisa ser notificada.

Automação: As tags são a base para a automação. Você pode usar a API do New Relic para, por exemplo, aplicar uma política de alerta a todas as entidades com a tag critical-service:true.

É importante distinguir tags de atributos customizados (adicionados via AddCustomParameter):

- **Tags:** São metadados da *entidade*. São de baixa cardinalidade (poucos valores distintos, como 'produção', 'staging'). São ideais para agrupar e filtrar as próprias entidades.
- **Atributos Customizados:** São metadados de um *evento* específico (uma transação, um erro). São de alta cardinalidade (muitos valores distintos, como `userId`, `productId`). São ideais para análises detalhadas dentro de um tipo de transação.

Uma estratégia de tagging bem planejada é a fundação para a observabilidade em escala. Ela transforma uma massa de dados de telemetria em um inventário organizado e navegável que reflete a lógica do seu negócio.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Característica	Tags	Atributos Customizados
Escopo	Entidade (Aplicação, Host)	Evento (Transação, Erro)
Cardinalidade	Baixa (Ambiente, Equipe)	Alta (ID do Usuário, ID do Pedido)
Uso Principal	Filtrar e agrupar entidades.	Filtrar e analisar eventos.

Figura 10 –
Fonte: Elaborado pelo autor (2026)

Coleta de Dados e Métricas

Apdex Score

O tempo de resposta médio, embora útil, pode ser uma métrica enganosa. Uma única transação extremamente lenta pode inflar a média, mascarando uma experiência que, para a maioria dos usuários, foi satisfatória. Para resolver essa ambiguidade, o New Relic utiliza o **Apdex (Application Performance Index)**, um padrão aberto que mede a satisfação do usuário com o tempo de resposta de uma aplicação.

O Apdex converte muitas medições de performance em um único número em uma escala de 0 a 1 (0 = todos os usuários estão frustrados; 1 = todos os usuários estão satisfeitos). O cálculo é baseado em um limiar de tempo de resposta definido pelo usuário, conhecido como **Apdex T**.

Com base neste limiar T, cada transação é classificada em uma de três categorias:

- **Satisfeita (Satisfied):** a transação foi concluída em um tempo menor ou igual a T. (Ex: T = 0.5s, tempo de resposta = 0.3s). O usuário está satisfeito.
- **Tolerante (Tolerating):** a transação foi concluída em um tempo maior que T, mas menor ou igual a 4 vezes T. (Ex: T = 0.5s, tempo de resposta = 1.5s). O usuário está tolerando a lentidão, mas não está feliz.

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

- **Frustrada (Frustrated):** a transação demorou mais de 4 vezes T para ser concluída, ou retornou um erro de servidor. (Ex: T = 0.5s, tempo de resposta = 2.5s). O usuário está frustrado.

A Fórmula do Apdex, o score Apdex é calculado da seguinte forma:

Apdex Score = (Número de Transações Satisfeitas + (Número de Transações Tolerantes / 2)) / (Número Total de Transações)

As transações frustradas não contribuem positivamente para o score. A fórmula efetivamente pondera as transações tolerantes como "meio satisfeitas". Por que o Apdex é Importante?

- **Foco no Usuário:** o Apdex traduz métricas de performance em uma medida de satisfação do usuário, que é o que realmente importa para o negócio.
- **Padrão Consistente:** ele fornece uma maneira uniforme de comparar a performance de diferentes aplicações, transações ou ambientes.
- **Alertas Inteligentes:** é muito mais eficaz alertar sobre uma queda no Apdex Score (ex: abaixo de 0.85) do que em um aumento no tempo de resposta médio, pois o Apdex é mais resiliente a outliers e reflete melhor um problema que afeta um grande número de usuários.

Escolher um valor de T apropriado é crucial. Ele deve ser um objetivo de performance realista para a aplicação ou transação específica. O valor padrão no New Relic é de 0.5 segundos para transações de APM e 7 segundos para transações de Browser, mas ele deve ser ajustado para refletir as expectativas dos seus usuários.

Categoria	Tempo de Resposta (t)	Impacto no Usuário	Contribuição para o Score
Satisfeita	t ≤ T	Satisfeito	1 ponto
Tolerante	T < t ≤ 4T	Tolerando	0.5 ponto
Frustrada	t > 4T ou Erro	Frustrado	0 pontos

Figura 11 – Valor de T
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

Coletar Métricas Customizadas

Enquanto o agente do New Relic coleta automaticamente uma vasta gama de métricas de performance (tempo de resposta, taxa de erros, uso de CPU), a verdadeira observabilidade é alcançada quando correlacionamos esses dados de sistema com os dados de negócio. As **métricas customizadas** são a principal ferramenta para injetar KPIs de negócio e métricas de aplicação específicas na plataforma New Relic.

Já introduzimos brevemente a API RecordMetric, mas aqui vamos aprofundar no conceito. Uma métrica no New Relic é um dado numérico, agregado ao longo do tempo. Diferente de um evento (como um Transaction ou um log), que representa uma ocorrência discreta com um rico conjunto de atributos, uma métrica é otimizada para armazenamento e consulta de dados de série temporal de forma eficiente.

O nome de uma métrica customizada deve seguir uma convenção hierárquica para facilitar a organização e a busca Custom/<Category>/<Name>, por exemplo *Custom/ShoppingCart/ItemsAdded*.

- Custom/: prefixo obrigatório para todas as métricas criadas pelo usuário.
- <Category>: uma forma de agrupar métricas relacionadas (ex: ShoppingCart, PaymentService).
- <Name>: o nome específico da métrica (ex: ItemsAdded, TransactionValue).

Quais os tipos de metricas cutomizadaS?

Métricas de Contagem (Count Metrics): usadas para contar ocorrências. Por exemplo, quantas vezes um item foi adicionado a um carrinho.

Métricas de Valor (Value Metrics ou Gauge Metrics): usadas para medir um valor em um ponto no tempo. Por exemplo, o valor total de uma transação de pagamento ou o número de itens em uma fila.

Por que usar Métricas Customizadas?

Monitorar KPIs de Negócio: rastrear métricas como Novos_Registros, Pedidos_Concluidos ou Valor_Vendido ao lado de métricas de performance como

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

tempo de resposta e taxa de erros. Isso permite responder a perguntas como: "A lentidão no checkout está impactando o número de pedidos concluídos?".

Monitorar Recursos da Aplicação: medir valores que o agente não captura por padrão, como o número de conexões ativas em um pool, o tamanho de um cache interno ou o número de mensagens em uma fila.

Eficiência: para dados de alta frequência, enviar métricas é muito mais eficiente em termos de ingestão de dados (e custo) do que enviar um evento customizado para cada ocorrência.

Em resumo, use **eventos customizados** quando precisar de detalhes sobre uma ocorrência individual (quem, o quê, quando). Use **métricas customizadas** quando precisar contar ou medir algo que acontece com frequência e você está mais interessado na tendência agregada ao longo do tempo.

Tipo de Dado	Natureza	Cardinalidade	Caso de Uso	Exemplo NRQL
Métrica	Agregado (série temporal)	Baixa	Contar eventos de alta frequência, medir valores.	`SELECT sum(newrelic.timeslice.value) FROM Metric`
Evento	Discreto (ponto no tempo)	Alta	Investigar uma ocorrência específica com contexto rico.	`SELECT * FROM CustomEvent`

Figura 12 – Métricas Customizadas
Fonte: Elaborado pelo autor (2026)

Sampling

Em sistemas com alto volume de tráfego, coletar e armazenar detalhes para cada transação, erro ou evento customizado pode se tornar proibitivamente caro e computacionalmente intensivo. Para gerenciar isso, o New Relic emprega uma técnica chamada **Sampling (Amostragem)** para certos tipos de dados, principalmente dados de **evento**.

É crucial entender a diferença:

- **Dados de Métrica (Metric Data):** são agregados no agente antes de serem enviados. Por exemplo, o agente não envia o tempo de cada uma das

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

10.000 transações que ocorreram em um minuto. Em vez disso, ele calcula min, max, average, sum e count e envia esse pequeno resumo. **Dados de métrica não são amostrados.**

- **Dados de Evento (Event Data):** são eventos individuais e detalhados. Transaction, TransactionError, Span e eventos customizados são exemplos. Cada evento contém um rico conjunto de atributos. **Dados de evento são sujeitos à amostragem.**

O agente do New Relic tem um buffer para armazenar eventos. A cada minuto (no "harvest cycle"), ele envia o conteúdo desse buffer para o New Relic. Se o número de eventos gerados em um minuto exceder a capacidade do buffer, o agente começa a tomar decisões de amostragem.

O New Relic usa **Adaptive Sampling**. O agente analisa o tráfego e tenta preservar os eventos mais importantes. Por exemplo, ele sempre tentará manter os traces de transações mais lentas e os que contêm erros, pois eles são mais valiosos para o diagnóstico.

Quando a amostragem ocorre, os dados de evento que você vê no New Relic são apenas um subconjunto representativo do total. Isso tem implicações diretas em suas queries NRQL:

- **SELECT count(*):** se você executar SELECT count(*) FROM Transaction, o resultado será o número de eventos *armazenados* no New Relic, não o número total de transações que sua aplicação realmente processou.
- **SELECT average(duration):** cálculos de média, percentis, etc., ainda são estatisticamente válidos, pois a amostra é projetada para ser representativa.

Para compensar a amostragem e estimar o valor real, o NRQL fornece a palavra-chave EXTRAPOLATE. Quando você usa EXTRAPOLATE, o New Relic analisa os metadados de amostragem enviados com os eventos e multiplica os resultados para estimar como seriam se 100% dos dados tivessem sido coletados.

Sempre que você estiver fazendo uma contagem (count, uniqueCount) em um tipo de dado de evento amostrado e quiser saber o número total estimado, não apenas a contagem na amostra. Para outros cálculos como average, min, max, percentile, o

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

EXTRAPOLATE não é necessário, pois eles são representativos mesmo com a amostra.

Entender a amostragem é fundamental para não tirar conclusões erradas dos seus dados. Lembre-se: se você está contando eventos, pergunte-se se precisa da contagem da amostra ou da contagem total extrapolada.

Query NRQL	Resultado
`SELECT count(*) FROM Transaction`	Número de eventos de transação na amostra.
`SELECT count(*) FROM Transaction EXTRAPOLATE`	Estimativa do número total de transações que ocorreram.
`SELECT count(*) FROM Metric`	Número total de transações que ocorreram.

Figura 13 – Amostragem
Fonte: Elaborado pelo autor (2026)

Retenção de Dados

A retenção de dados refere-se ao período de tempo durante o qual o New Relic armazena os diferentes tipos de dados de telemetria que coleta. Compreender as políticas de retenção é crucial para saber por quanto tempo você pode realizar análises históricas, investigar incidentes passados e identificar tendências de longo prazo.

A política de retenção não é única para todos os dados; ela varia significativamente com base no **tipo de dado** e no **plano de assinatura** da sua conta.

O New Relic otimiza o armazenamento (e o custo) tratando diferentes tipos de dados de maneiras distintas.

Dados de Evento (APM, Browser, Mobile, Custom Events): por serem detalhados e volumosos, geralmente têm um período de retenção mais curto. No plano Standard, a retenção padrão para a maioria dos eventos é de **8 dias**. Isso significa que você pode investigar os detalhes de uma transação específica que ocorreu na última semana, mas não no mês passado.

Dados de Métrica (Metric Data): por serem pré-agregados e mais compactos, eles podem ser armazenados por mais tempo. A retenção para dados de métrica é significativamente maior, geralmente de **13 meses** ou mais. Isso permite a análise de

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

tendências de longo prazo, como comparar a performance da Black Friday deste ano com a do ano anterior.

Dados de Trace (Spans): os detalhes de traces distribuídos (spans) também são volumosos e, portanto, têm uma retenção curta, semelhante à dos eventos (geralmente 8 dias).

Logs: a retenção de logs é altamente configurável e depende do plano, mas a retenção padrão no plano Standard é de **30 dias**.

Os planos de assinatura do New Relic afetam diretamente a retenção.

- **Free Tier:** possui os períodos de retenção mais curtos.
- **Standard:** oferece os períodos de retenção padrão descritos acima (ex: 8 dias para eventos).
- **Pro / Enterprise (com Data Plus):** esses planos oferecem a opção de **retenção estendida**. Com o add-on "Data Plus", as organizações podem aumentar a retenção de dados de evento para 30, 60, 90 dias ou até mais, dependendo das suas necessidades de conformidade e análise.

Saber os períodos de retenção evita surpresas. Se um gerente pedir um relatório sobre a performance detalhada de uma transação de 3 meses atrás, você precisa saber que, em um plano padrão, os dados de evento para essa análise não existem mais. No entanto, os dados de *métrica* (como tempo de resposta médio, throughput) para aquele período ainda estarão disponíveis. Você pode dizer "como" a aplicação estava performando em geral, mas não pode mais ver os "porquês" detalhados de uma única transação.

Tipo de Dado	Retenção Padrão (Plano Standard)	Caso de Uso da Análise
Eventos (Transaction, etc.)	8 dias	Investigação de incidentes recentes, depuração.
Métricas (Metric)	13+ meses	Análise de tendências sazonais, planejamento de capacidade.
Traces (Span)	8 dias	Análise detalhada de uma requisição distribuída específica.
Logs	30 dias	Auditoria de segurança, análise de causa raiz de erros.

Figura 14 – Retenção de Dados
Fonte: Elaborado pelo autor (2026)

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, abordarmos os **fundamentos de observabilidade e APM**, detalhando a arquitetura da plataforma New Relic e sua estrutura de contas, usuários e chaves de API. O foco inicial recaiu sobre o **agente APM**, explorando seu ciclo de coleta, a distinção entre transações web e de background, e o equilíbrio entre a **instrumentação automática e a customizada**. O conteúdo ensinou como refinar o monitoramento através de atributos, APIs do agente e métricas personalizadas para uma visibilidade precisa do software.

Na segunda parte, exploramos o **modelo de entidades** e as interdependências entre aplicações, infraestrutura e serviços. A aula capacita o(a) aluno(a) a navegar pelo ecossistema utilizando o **Entity Explorer** e a monitorar a saúde do ambiente com as ferramentas **Navigator e Lookout**. Por fim, o uso do **Service Map** é apresentado como o recurso essencial para a visualização clara das dependências e da topologia do sistema.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

REFERÊNCIAS

NEW RELIC. **Application Performance Monitoring (APM) Overview**. Disponível em: https://docs.newrelic.com/docs/apm/. Acesso em: 27 jan. 2026.

NEW RELIC. **Instrumentation and Tracing.** Disponível em: https://docs.newrelic.com/docs/apm/agents/. Acesso em: 27 jan. 2026.

NEW RELIC. **NerdGraph API Documentation.** Disponível em: https://docs.newrelic.com/docs/apis/nerdgraph/. Acesso em: 27 jan. 2026.

NEW RELIC. **New Relic .NET Agent Documentation.** Disponível em: https://docs.newrelic.com/docs/apm/agents/net-agent/. Acesso em: 27 jan. 2026.

NEW RELIC. **NRDB – New Relic Database**. Disponível em: https://docs.newrelic.com/docs/data-apis/. Acesso em: 27 jan. 2026.

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com

PALAVRAS-CHAVE
APM, Instrumentação, Observabilidade

FIAP

PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com



PDF exclusivo para Gabriel Pinelli Silva - rm373763
gabrieltocaccelli@gmail.com